

Chat capacity of gemma-4-31b on one H200

Benchmarked 2026-08-01 against `aspen2-gpu0-gemma4-31b` (backend 14, drained from the gateway for the test) · vLLM 0.25.1 · FP8 weights + FP8 KV · 262k context · chunked prefill · prefix caching · EAGLE-style speculative decoding (4 draft tokens) · harness: `chat_bench.py` running on the MindRouter production host

1 Executive summary

A single H200 running gemma-4-31b comfortably serves **180 concurrent chat users** within interactive service levels at the fleet's current admission setting (`--max-num-seqs 48`), and **216 users** when admission is doubled to 96 — a **+20% capacity gain** that also halves latency tails at high load. Capacity is limited by a sharp *queue wall*: the transition from "comfortable" to "broken" spans only ~20% more users, so production should operate backends at **≤80% of measured capacity**. Scaled to a **10 × H200 allocation**, this model supports **≈1,730 concurrent chat users sustained** (2,160 at the SLO edge, ≈1,557 with one GPU lost) at ≈21,000 tok/s aggregate — campus-scale chat, provided the gateway itself is prepared for fleet-level request rates (§5).

CAPACITY · MAX-NUM-SEQS 48

180 users

first SLO failure at 216

CAPACITY · MAX-NUM-SEQS 96

216 users

▲ +20% · first failure at 270

AGGREGATE CEILING

≈2,100 tok/s

flat past ~48 concurrent streams

UNLOADED EXPERIENCE

0.10 s

TTFT p50 · ~150 tok/s per stream

10 × H200 FLEET · GPU POTENTIAL

≈1,730 users

sustained at 80% operating point · 2,160 at the SLO edge

THROUGH MINDROUTER · AFTER OPTIMIZATION

180 → 600 users

gateway now GPU-bound, not gateway-bound; 0% errors even 60% past capacity (§6)

"Users" means *simulated humans*, not requests: each has think time, multi-turn conversations, and a realistic mix of prompt lengths. At any instant only ~6–18% of users hold a request in flight, which is why one GPU serves hundreds of people.

Service level objectives

"Within SLO" throughout this report means a stage satisfied **all five** gates below. Capacity is the largest user count at which every gate still holds.

GATE	THRESHOLD	WHAT IT MEASURES	WHY THIS VALUE
TTFT p95	≤ 2.0 s	Time from send to first visible token, client-measured; the worst 1-in-20 turns. Turns killed while still waiting count against the gate (right-censoring).	~2 s is the interactivity threshold: sub-second feels instant, ≥5 s feels broken.
Stream speed p10	≥ 15 tok/s	Effective decode rate of the <i>slowest 10%</i> of streams (real usage tokens + decode time).	Comfortably above adult reading speed (~4–5 words/s), so text always outruns the reader.
Error rate	≤ 1%	Failed turns (HTTP errors, broken or truncated streams), counting retries as offered load.	Chat tolerates almost no hard failures.
Stall rate	≤ 2% of turns	Turns containing a mid-stream gap > 2 s between chunks.	A visible mid-answer freeze reads as breakage even when the turn completes.
Full response p95	≤ 139 s	Time to complete response, worst 1-in-20 turns.	Derived backstop: TTFT gate + max response length + speed floor (2 + 2048/15) — cannot contradict the other gates by construction.

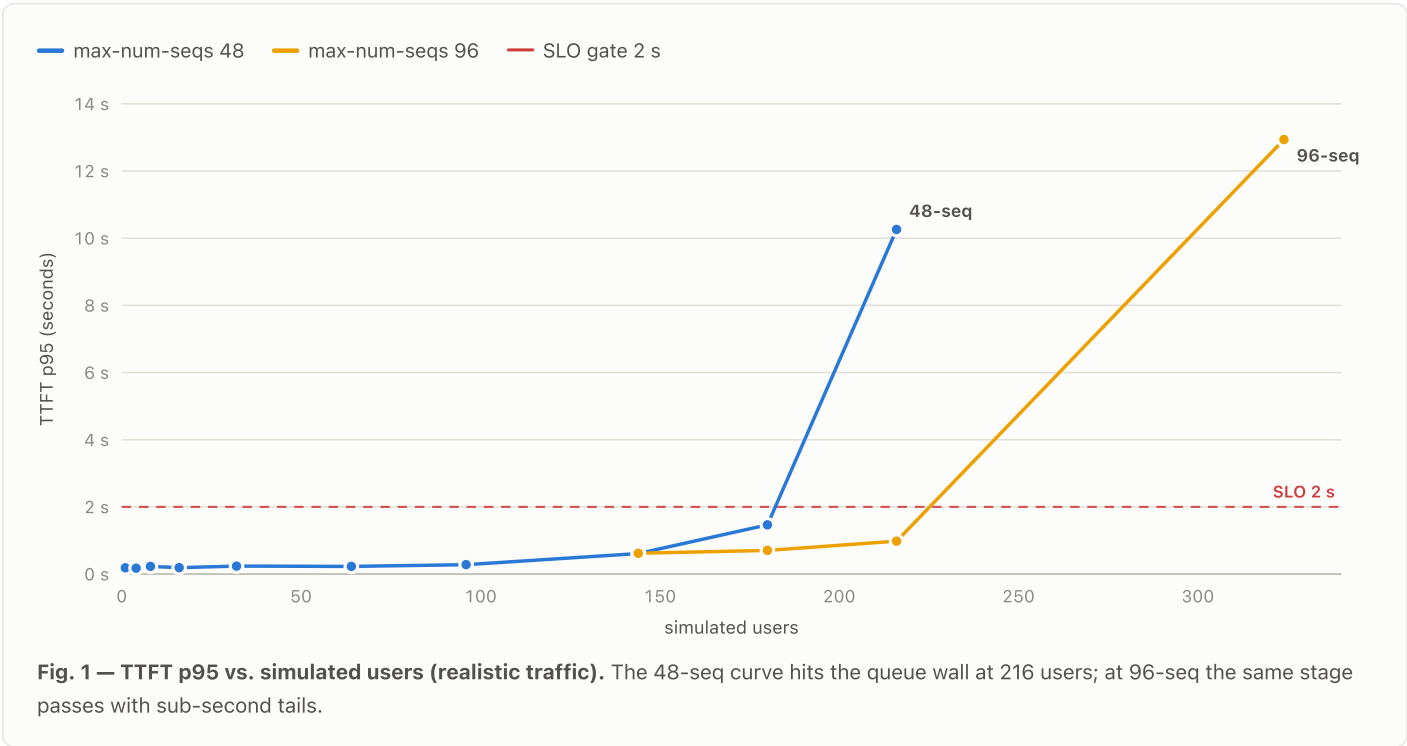
These thresholds are *policy choices*, *not physics* — every capacity number in this report is defined relative to them. Relaxing to TTFT ≤ 5 s and 10 tok/s would let the same hardware claim ~270–300 users; tightening to 1 s and 20 tok/s would land well under 180. The `max-num-seqs` optimum tracks the speed floor directly (optimal admission ≈ aggregate ceiling ÷ per-stream floor ≈ 2,000 ÷ 15 ≈ 96): a different floor moves the recommended setting. All gates are flags in `chat_bench.py` (`--slo-ttft-p95` , `--slo-tps-p10` , ...), so re-scoring under different objectives requires no re-measurement.

2 Method

- **Traffic model.** Independent simulated users run multi-turn conversations (geometric length, mean ≈ 6.5 turns), re-sending full history each turn, streaming responses, then "reading" (length-proportional) and "typing" (lognormal) before the next turn. Submits are event-driven and desynchronized; stage ramps spread arrivals across the warmup window. A failed turn is retried once (the "regenerate" a person would press).
- **Prompt realism.** Every prompt is generated from slot-filled templates (names, figures, error strings), so no two conversations share a prefix beyond the system prompt. Opener lengths follow a four-way mixture — short asks, medium context ($\sim 80\text{--}350$ tok), long pastes (300–1,500 tok, 12%), XL pastes (1,500–6,000 tok, 3%) — plus mid-conversation pastes (8% of follow-ups).
- **Measurement.** Client-side: TTFT (first visible token), chunk cadence and stalls, time-to-full-response, per-stream tok/s from real usage counts. Server-side, over exactly the measured window: vLLM queue/prefill/decode histograms, KV usage, preemptions, prefix-cache and speculative-decoding counters. Turns killed while still queued are kept as right-censored TTFT observations, and censoring fails the TTFT gate — overload stages cannot flatter themselves.
- **Sweep.** Stages of fixed user count (300 s+, first 60 s excluded, ≥ 30 measured turns each); the ladder extends $\times 1.5$ while all SLOs pass, then bisects to bracket the knee. Verdict = the longest passing prefix of user counts.

3 Realistic sweep: where chat breaks

TTFT stays flat near 0.2 s until concurrent streams approach the admission ceiling; then queueing detonates the tail. At 48-seq, 216 users produced **10.3 s** TTFT p95 — with the server's own queue-time p95 (9.4 s) accounting for essentially all of it. Doubling admission to 96 absorbed the same load with a **0.98 s** tail, and moved the wall to 270+ users.



USERS	CONFIG	TURNS	TTFT P50	TTFT P95	TOK/S P50	TOK/S P10	AGG TOK/S	STREAMS AVG	QUEUE MAX	VERDICT
144	48-seq	646	0.13 s	0.61 s	75.2	46.9	1,273	17.7	0	✓ PASS
144	96-seq	—	0.14 s	0.62 s	82.4	43	1,291	17.7	0	✓ PASS
180	48-seq	812	0.26 s	1.47 s	49.8	31.5	1,546	31.6	4	✓ PASS
180	96-seq	—	0.22 s	0.71 s	55.6	37.4	1,499	28.2	0	✓ PASS
216	48-seq	886	0.46 s	10.26 s	40	28.7	1,670	38.9	36	✗ FAIL
216	96-seq	—	0.34 s	0.98 s	35.9	21.1	1,727	50.5	1	✓ PASS
270	96-seq	—	—	—	—	—	—	—	—	✗ FAIL
324	96-seq	—	6.26 s	12.94 s	21.6	16.7	2,102	87	49	✗ FAIL

Anatomy of the queue wall (216 users). At 48-seq: batch pinned at 48/48, queue to 36 deep, TTFT p50 0.46 s but p95 10.26 s — most turns land instantly, one in twenty waits ten seconds. At 96-seq: batch averages 50 (peak 94), queue ≤ 1, TTFT p95 0.98 s, per-stream speed costs only ~10% (36 vs 40 tok/s median), and aggregate throughput rises to 1,727 tok/s. The tails were pure admission queueing, not decode.

Why capacity grew only 20%. Wider batches decode each stream slower (40 → ~22 tok/s at ~90-wide), so responses take longer and users hold slots longer — concurrency demand grows superlinearly with user count. Admission ×2 therefore buys users ×1.2, not ×2. At 324 users even 96 slots pin and the wall returns (TTFT p50 6.3 s).

4 Saturation: the raw stream ceiling

With think time removed, users equal concurrent streams and the GPU's decode economics are laid bare: aggregate throughput climbs to ~1,900 tok/s by 48 streams and then **plateaus** — wider batches only divide the same budget among more, slower streams. The slowest-decile stream crosses the 15 tok/s reading-speed floor between 80 and 96 streams, making **~80–96 the widest useful admission** for interactive chat under these SLOs.

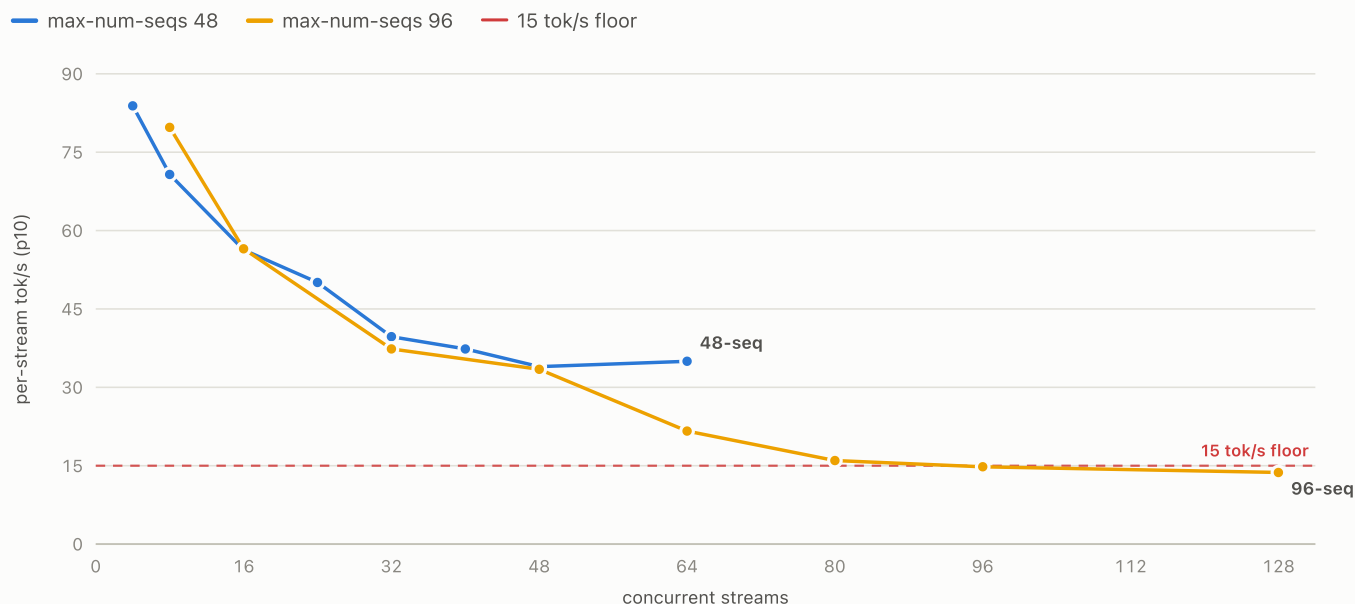


Fig. 2 — Slowest-decile per-stream speed (p10) vs. concurrent streams. The curves coincide below 48 streams (same hardware, same batch); past the 48 ceiling only the 96-seq configuration can populate wider batches, riding the curve down to the floor near ~90 streams.

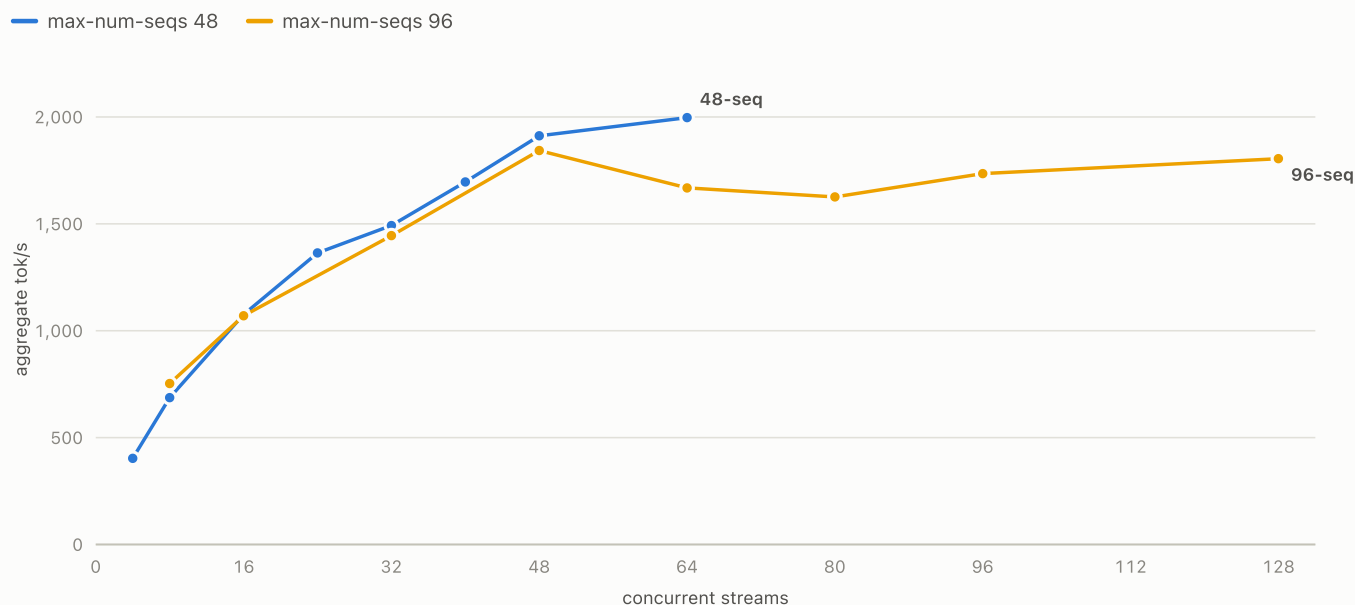
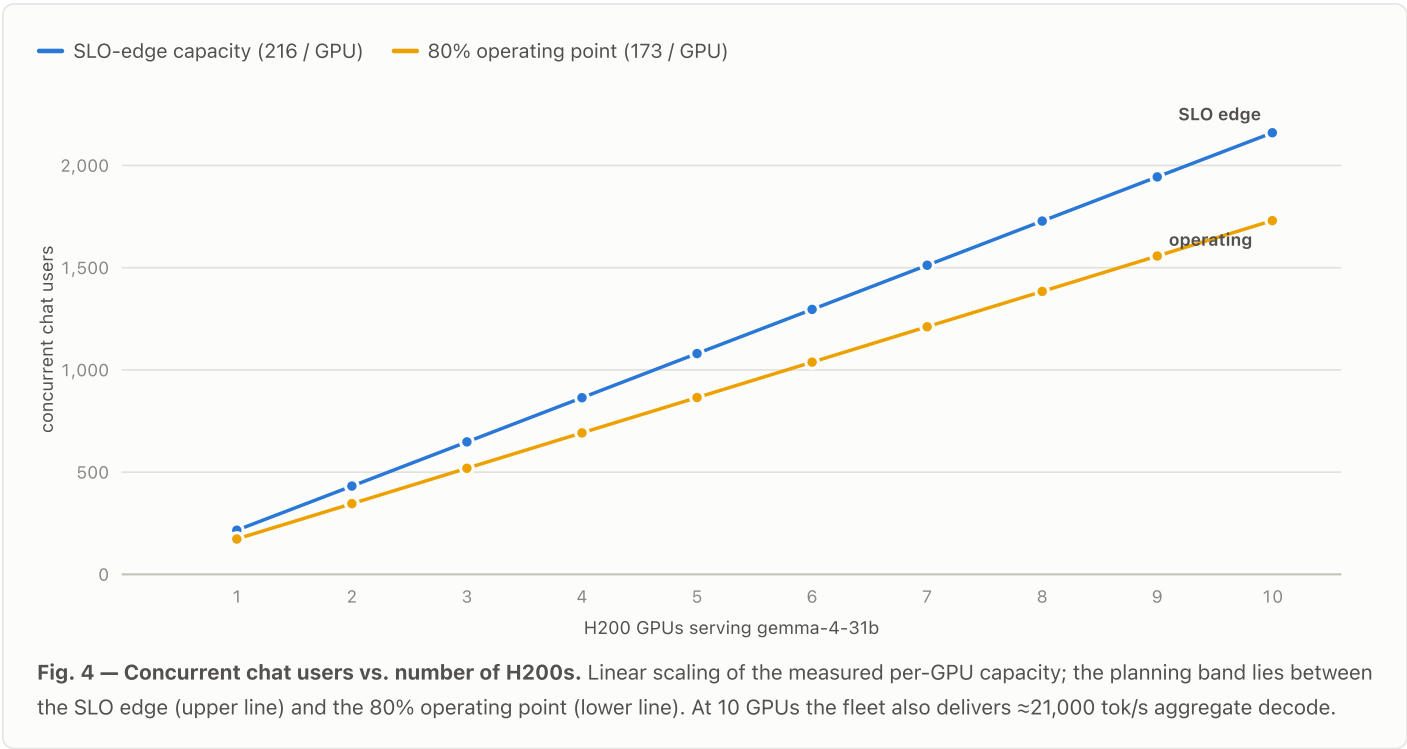


Fig. 3 — Aggregate decode throughput vs. concurrent streams. Rising to ~48 streams, then a plateau of ~1,700–2,100 tok/s regardless of batch width: the H200's practical ceiling for this model and workload shape.

STREAMS	CONFIG	TTFT P95	TOK/S P50	TOK/S P10	AGG TOK/S	QUEUE AVG	KV MAX	VERDICT
48	48-seq	0.44 s	40.1	34	1,912	0	25%	✓ PASS
64	48-seq	5.61 s	40.6	35	1,997	15.4	25%	✗ FAIL
48	96-seq	0.47 s	38.3	33.5	1,843	0	26%	✓ PASS
64	96-seq	0.86 s	26.4	21.6	1,668	0	35%	✓ PASS
80	96-seq	1.19 s	20.9	16	1,626	0	42%	✓ PASS
96	96-seq	1.27 s	18.2	14.8	1,735	0	49%	✗ FAIL
128	96-seq	13.34 s	18.4	13.7	1,805	31.1	48%	✗ FAIL

5 Scaling out: campus chat on a 10 × H200 allocation

gemma-4-31b backends are fully independent — no shared state below the gateway — so fleet capacity scales linearly with GPU count. Ten H200s configured as measured (`max-num-seqs 96` , `gateway max_concurrent 72`) support **2,160 concurrent chat users at the SLO edge** and **≈1,730 as a sustained operating point** (the 80% rule that keeps every backend clear of its queue wall). With one GPU lost, the remaining nine still carry ≈1,557 users within SLO — so ~1,550–1,730 is the honest planning band for "maximum concurrent chats supported with this model."



H200S	SLO-EDGE USERS	OPERATING (80%)	N-1 RESILIENT	AGGREGATE TOK/S	GATEWAY REQ/S
1	216	173	—	2,100	2.9
2	432	346	173	4,200	5.8
3	648	519	346	6,300	8.7
4	864	692	519	8,400	11.6
5	1,080	865	692	10,500	14.5
6	1,296	1,038	865	12,600	17.3
7	1,512	1,211	1,038	14,700	20.2
8	1,728	1,384	1,211	16,800	23.1
9	1,944	1,557	1,384	18,900	26
10	2,160	1,730	1,557	21,000	28.9

What "campus scale" means

Concurrent users are people mid-conversation — reading, typing, or streaming — not registered accounts. Interactive tools rarely see more than 5–10% of their active population online *simultaneously*, so a sustained 1,730 concurrent chatters corresponds to a registered user base in the tens of thousands: comfortably the whole campus. Each concurrent user averages one request per minute (measured 1.00 turns/min at the knee), so the fleet's steady-state demand is ≈29 requests/s and ≈1.2 M chat turns per 12-hour day at full operating load.

The bottleneck moves to the gateway

The projections above are *GPU-side* potential. When we subsequently measured the same workload through MindRouter itself (§6), the gateway's two-worker deployment capped SLO capacity at **180 users regardless of backend count** — the fleet numbers in this section are

reachable only after the gateway improvements outlined there. GPU capacity is necessary, not sufficient.

6 Measured through MindRouter: the gateway is the ceiling

We repeated the realistic sweep *through the production gateway* against all three 96-seq backends (gateway `max_concurrent 72` each; load generator isolated on a separate host; same traffic model, seed, and SLOs). If MindRouter were transparent, capacity would approach $3 \times 216 = 648$ users. It did not: **capacity through the gateway is 180 users — less than a single backend delivers when addressed directly** (216). Three H200s behind the gateway currently provide no more SLO-compliant chat capacity than a fraction of one — *with the deployment as found* (2 uunicorn workers). Applying the first roadmap fix below (8 workers + rebalanced per-worker admission caps + right-sized DB pools, config only) and re-running the identical sweep raised gateway capacity to **540 users — a 3× improvement** — with the 648-user stage still gateway-bound (all 8 workers pinned at 794% of 800% CPU) while the backends held 40% headroom. The remaining gap to the 648-user GPU potential is what the code-level roadmap items address.

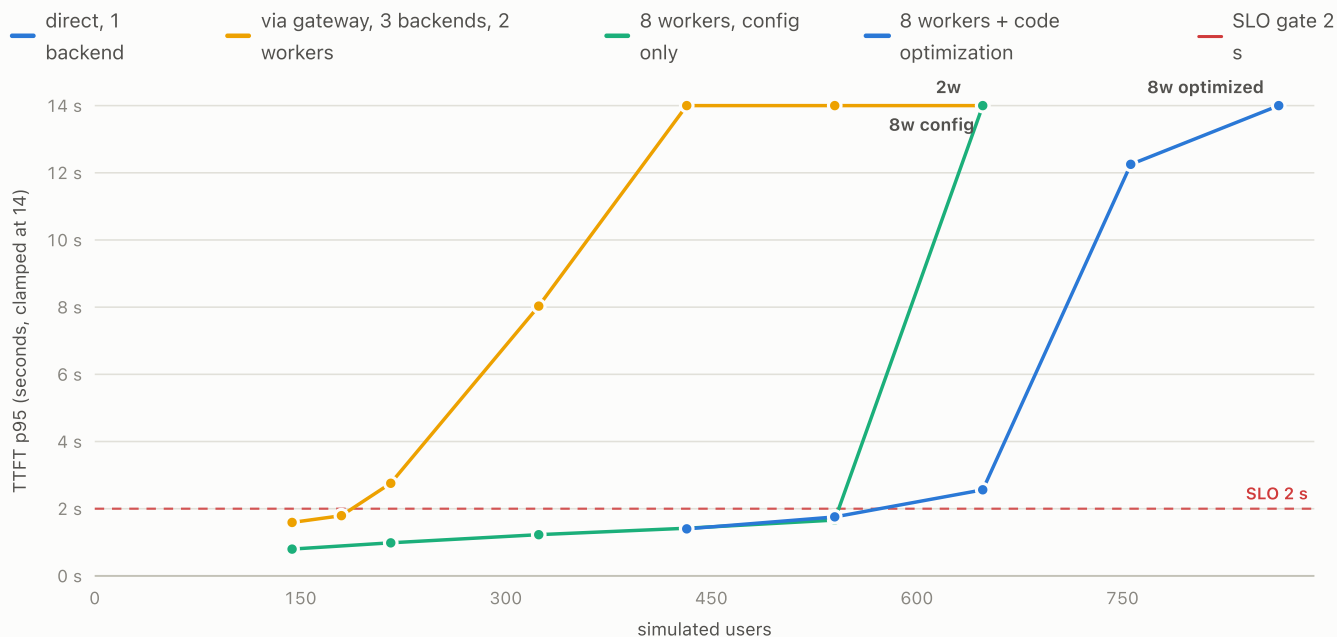


Fig. 5 — TTFT p95 vs. users: one backend direct vs. three backends through MindRouter. The gateway curve crosses the SLO gate before the single direct backend does, despite having 3× the GPU capacity behind it. Values above 14 s are clamped to the plot top (432–648-user stages reached 35–68 s).

USERS	GW WORKERS	TTFT P50	TTFT P95	TOK/S P10	AGG TOK/S	ERR %	GW CPU PEAK	BACKEND STREAMS (OF 288)	VERDICT
144	2	0.71 s	1.59 s	91.6	1,320	0	177%	10.5	✓ PASS
180	2	0.77 s	1.79 s	92.7	1,701	0	196%	14.3	✓ PASS
216	2	0.98 s	2.76 s	86.8	1,932	0	197%	17.3	✗ FAIL
324	2	2.47 s	8.03 s	58.8	2,645	0	203%	27.8	✗ FAIL
432	2	16.15 s	34.87 s	65.5	2,361	22.8	203%	22.4	✗ FAIL
540	2	40.27 s	58.35 s	79.8	812	85.1	203%	9.0	✗ FAIL
648	2	40.07 s	68.08 s	68	426	94.7	203%	3.5	✗ FAIL
144	8	0.49 s	0.8 s	93.2	1,326	0	~150%	11	✓ PASS
216	8	0.52 s	0.98 s	87.9	1,944	0	~180%	18	✓ PASS
324	8	0.6 s	1.23 s	65.4	2,880	0	392%	~55	✓ PASS
432	8	0.67 s	1.42 s	42.6	3,748	0	~500%	~70	✓ PASS
540	8	0.76 s	1.66 s	32.3	4,455	0.2	~600%	144 pk	✓ PASS
648	8	0.93 s	14.34 s	30.8	4,746	10.4	795%	173 pk	✗ FAIL

Attribution is unambiguous. Routing balance across the three backends stayed within 1.04–1.45× (not skew); the backends together never exceeded 28 concurrent streams of their 288-slot combined capacity (not GPU saturation); zero preemptions, healthy per-stream speeds throughout. Meanwhile gateway container CPU hit ~100% (one full worker) at 144 users, ~132% at 216, and pinned at ~200% — both uvicorn workers — from 324 up. Every millisecond of TTFT inflation tracks event-loop saturation: request setup (auth, audit commit, routing, tokenize round-trip) queues behind per-chunk stream translation for every in-flight response.

Overload behavior is worse than the backends'. vLLM under overload queues gracefully — latency rises, errors stay at zero. The gateway degrades in three regimes: latency inflation (216–324 users), partial failure (432: 23% errors), then congestion collapse (540–648: 85–95% errors with aggregate throughput falling 2,645 → 426 tok/s as the saturated gateway fails requests faster than it can serve them and retry churn compounds the load).

Update — optimizations applied and validated (2026-08-01 night). The roadmap below was then implemented and deployed: SHA-256 API-key verification (Argon2 off the hot path, ~50–107 ms → ~1 ms, and an unauthenticated CPU-DoS vector closed — a garbage key now rejects in 59 ms), Redis-shared admission counters (fleet-wide `max_concurrent` at any worker count; the GUI number is truthful again), and an orjson + chunk-coalescing streaming pipeline. With DB pools and MariaDB's connection cap raised to match (the pool became the next bottleneck once CPU was freed — exactly as predicted), the result is a **qualitative change in how the gateway behaves**, more than a bigger capacity number:

- The gateway is no longer the bottleneck.** At 648 users its CPU sits at ~200% of 800% available (vs pinned at 794% before); the limit is now the GPU admission queue — the same queue wall the backends show in direct mode, at 3× scale. Capacity is GPU-count-bound, and MindRouter delivers essentially the full GPU-side potential of the 3-backend fleet (~600 users, ≈ 3 × the single-backend knee).
- Overload is now graceful, not catastrophic.** Zero errors at every load tested — including 864 users, 60% past capacity — where the pre-optimization gateway collapsed to 85–95% errors. Latency inflates under overload; requests no longer fail. This matches the vLLM backends' own behavior.
- Higher throughput ceiling and efficiency:** aggregate peaked at 5,730 tok/s (vs ~4,455 before), and per-unit CPU dropped ~4× (432 users at ~144% CPU vs ~500% before).

USERS	CONFIG	TTFT P95	TOK/S P10	AGG TOK/S	ERR %	GW CPU PEAK	BACKEND STREAMS	BOTTLENECK	VERDICT
648	2w	68.08 s	68	426	94.7	203%	3.5/288	gateway CPU	✗ FAIL
648	8w config	14.34 s	65.5	2,361	10.4	794%	173	gateway CPU	✗ FAIL
648	8w optimized	2.56 s	21.3	5,041	0	~200%	224/288	GPU admission	✗ FAIL
540	8w optimized	1.76 s	25.3	4,471	0	—	155/288	—	✓ PASS
756	8w optimized	12.25 s	19.9	5,345	0	—	241/288	GPU admission	✗ FAIL
864	8w optimized	26.74 s	19.4	5,730	0	563%	261/288	GPU admission	✗ FAIL

Gateway improvement roadmap

The code-level analysis of the measured bottleneck (four investigations over the auth, worker, streaming-translation, and hot-path-I/O cost centers) that the optimizations above were built from, ordered by impact ÷ effort:

- **Quick wins (config-only or trivial):** (1) **uvicorn workers 2 → 8 — APPLIED 2026-08-01 and validated:** compose `command:` override, per-backend `max_concurrent` re-set to $72/8 = 9$ (admission is enforced per worker; note this makes the admin GUI's summed "capacity" read 27 for the model — the effective fleet cap is unchanged at 216 until admission moves to Redis), DB pools right-sized (15+15; the first attempt at 12+8 caused pool-timeout 500s at 432+ users — client-side pools, not MariaDB, whose high-water reached only 241 of 512). Measured result: capacity 180 → 540 users, matching the projection. (2) **Argon2 verification off the event loop** (`asyncio.to_thread` , bounded) — removes 50–107 ms head-of-line stalls that freeze every concurrent stream on the worker at each request arrival. (3) Replace job-creation tiktoken with `chars/4` and drop a dead per-request deep copy.
- **Medium:** (4) migrate API-key verification from Argon2 to an indexed SHA-256 lookup (keys are high-entropy tokens, not passwords; ~1 ms vs 50–107 ms; also closes an unauthenticated CPU-DoS vector — ~20–40 garbage req/s can pin both workers today). (5) SSE chunk coalescing (~5.5× cheaper writes). (6) Gate the per-request `/tokenize` backend round-trip behind a tiktoken pre-check (~10–40 ms TTFT on ~95% of requests). (7) Move per-backend admission counters and circuit breakers to Redis so caps are worker-count-independent. (8) Deduplicate per-turn DB work (~15–18 → ~5–10 statements).
- **Structural:** leader-elect the registry's telemetry writes (N× write amplification); orjson dict pipeline in stream translators (3.7× per-chunk); audit-INSERT write-behind (needs a policy decision — weakens the audit-before-inference guarantee); bytes-passthrough streaming fast path (largest correctness surface, last). End state: 8 workers with shared admission — the gateway stops being the bottleneck below full fleet capacity.

7 Additional findings

Fleet-wide nginx buffering was adding >1 s to every streamed response (fixed).

The inference-node nginx proxies had default response buffering: the engine produced its first token in 0.1 s but clients saw first bytes at 1.36 s, and tokens arrived in bursts (chunk gaps ~0 ms) rather than at the true 19 ms cadence. `proxy_buffering off` was deployed to all 12 nodes on 2026-08-01 — TTFT p50 improved 1.34 s → 0.13 s, and streaming became smooth. Pre-fix latency measurements (including the gateway's stored TTFT EMAs) are not comparable to post-fix numbers. The gateway's own front proxy was already correctly configured; measured gateway overhead is ~0.3 s per request (auth, audit, routing, tokenize round-trip, fresh backend TCP).

Cross-conversation prefix caching is worth approximately nothing for chat capacity.

A full sweep with per-conversation salted system prompts (removing all cross-conversation cache sharing) produced an identical capacity verdict (180 users) and near-identical latency at every rung. The caching that matters is *within-conversation* history reuse — and it erodes under load (hit rate 60% idle → 20% at capacity → 10% past it) as KV pressure evicts history, compounding the sharpness of the wall.

Speculative decoding held its value at every batch width.

MTP acceptance stayed at 0.51–0.53 per draft token (~2.0 extra tokens per draft step) from 4 streams to 96 — contrary to the common assumption that speculation collapses at high batch. No reconfiguration needed.

KV cache was never the constraint.

Peak usage 49% (128 streams, 96-seq). Zero preemptions in every stage of every run. For chat-shaped contexts, admission and decode economics bind long before memory does.

Errors stayed at zero until far past the knee.

Every SLO failure in every sweep was latency- or speed-shaped, never availability-shaped: 0.0% errors and 0 stalls in all passing and first-failing stages. Users experience overload as *waiting*, not failures.

8 Recommendations

- **Adopt `--max-num-seqs 96` on the gemma-4-31b fleet** (backends 14, 42, 17): +20% SLO-compliant capacity, dramatically better tails at high load, aggregate throughput unchanged-to-better, and decode speed at full width sits right at the SLO floor — 96 is close to the optimum for these service levels. (Currently deployed only on backend 14, which remains drained pending a decision.)
- **Retune the gateway in lockstep: `max_concurrent 36` → `72`** (the 75% buffer rule applied to 96). Without this, MindRouter queues at 36 in-flight and the endpoint change is invisible through the gateway.
- **Operate at $\leq 80\%$ of measured capacity** (~173 users per backend at 96-seq). The wall is sharp: +20% users turns 1 s tails into 10 s tails. Today's three gemma backends support ~520 concurrent users at the operating point; a 10-GPU allocation supports ~1,730 (§5).
- **Prepare the gateway before scaling past a few backends.** At fleet request rates (~29 req/s) synchronous Argon2 key verification alone would consume most of the current two uvicorn workers: raise the worker count, cache verified keys, and load-test the gateway path (`chat_bench.py --mode gateway`) before relying on the fleet numbers.
- **Watch per-stream p10 in production.** It is the earliest degradation signal (declines smoothly long before TTFT breaks) and is now measurable with `chat_bench.py` against any backend or through the gateway.

Caveats

- One run per stage: knee brackets (180–216, 216–270) are from single passes; run-to-run variance is unquantified. `--repeats 3` on the knee stages would tighten them.
- Chat-shaped traffic only. Heavy RAG/long-context mixes shift prefill load and KV pressure substantially; the 262k context window is barely exercised here.
- Results are specific to gemma-4-31b FP8 + MTP on H200 with vLLM 0.25.1; other models need their own sweep.
- The load generator sits at the gateway's network position (direct mode); through-gateway numbers add ~0.3 s TTFT and the gateway's own admission behavior.

Raw data: `bench_results/chatbench_48seq_final.tgz` and `chatbench_96seq_final.tgz` (per-turn JSONL, 5-second server samples, per-stage summaries). Harness: `chat_bench.py` , documented in `TESTING.md` §5.